

A First Look at Multimodal Mobile Application Performance on XR and AI Smart Glasses

Majd Khalaf

Northeastern University
Boston, Massachusetts, USA

Mayank Chadha

Northeastern University
Boston, Massachusetts, USA

Mathew Garcia

Northeastern University
Boston, Massachusetts, USA

Mallesham Dasari

Northeastern University
Boston, Massachusetts, USA

Abstract

Smart glasses are the next mobile platform. Unlike phones, glasses operate as sensing-first devices that route data through a multi-hop wireless pipeline: a constrained glass to companion puck or a phone short-range edge link and cloud AI backends. This architecture exposes an entirely new class of wireless and mobile system problems that the community has not yet confronted. We present the first in-depth measurement study across several glasses and core XR and AI applications. Our findings open up rich problems for the community in rethinking glass-phone-cloud architecture and reimagining how smartphones can serve as efficient edge servers for the next generation of wearable computing.

CCS Concepts

• **Human-centered computing** → **Ubiquitous and mobile computing**; **Mixed / augmented reality**; • **Networks** → **Network performance analysis**; **Network measurement**; • **Hardware** → *Platform power issues*.

Keywords

smart glasses, XR, mobile applications, network performance, multimodal applications

ACM Reference Format:

Majd Khalaf, Mathew Garcia, Mayank Chadha, and Mallesham Dasari. 2026. A First Look at Multimodal Mobile Application Performance on XR and AI Smart Glasses. In *The 32nd Annual International Conference on Mobile Computing and Networking (MobiCom '26)*, October 26–30, 2026, Austin, TX, USA. ACM, New York, NY, USA, 18 pages. <https://doi.org/10.1145/3795866.3844477>



This work is licensed under a Creative Commons Attribution 4.0 International License.

MobiCom '26, Austin, TX, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2505-0/2026/10

<https://doi.org/10.1145/3795866.3844477>

1 Introduction

“If you can’t measure it, you can’t improve it.”

— Peter Drucker

Extended Reality (XR) and AI smart glasses are emerging as one of the most ambitious extensions of computing, pushing sensing, connectivity, and intelligence into lightweight, near-eye form factors. While the past decade has been dominated by bulky XR headsets (e.g., Apple Vision Pro [10], Meta Quest 3 [48]), recent advances in low-power display, camera, audio sensors, and cloud-based AI services have shifted the trend toward all-day wearable glasses [24].

Smart glasses already support popular applications such as live streaming, AI interactions, and real-time video conferencing [50]. With millions of glasses sold and user adoption witnessed [28], the industry’s multi-billion-dollar investment trajectory has begun shifting away from traditional AR/VR headsets toward glass platforms [63, 67].

Today’s glasses fundamentally rely on a co-located companion device, more commonly the user’s phone, for connectivity as well as computing resources. All glass interactions with Internet services traverse this companion device, creating a multi-stage pipeline involving glass to phone and phone to server, followed by responses from server to phone and phone to glass links. The links between glass and phone rely on a mix of Bluetooth Classic, BLE, or Wi-Fi Direct to communicate with the phone. Our central systems observation is that application performance cannot be explained by the G-UL alone: sensing, short-range transport, companion-phone processing, and cloud/application services form a coupled pipeline. In this pipeline, the phone often acts as an implicit edge server that relays, buffers, transforms media, and coordinates cloud interactions.

Despite this rapidly evolving ecosystem, we still lack a clear understanding of *how these new classes of workloads behave across the entire end-to-end pipeline spanning the glass, companion device, and the cloud*. This raises fundamental questions for mobile and networking systems: How do emerging AI and video streaming applications on glasses reshape network traffic patterns? How much load do glasses impose

on companion devices? How much of the application’s quality of experience (QoE) depends on the glass form factor, its edge companion device, and on the cloud resources given that there are a wide variety of glasses with different capabilities and price points in the market? Finally, how does the QoE of the same application differ when accessed directly on a phone vs. smart glasses? We aim to shed light on these questions through a comprehensive, end-to-end measurement study on a variety of smart glasses.

Study and technical challenges. We answer these questions through a comprehensive measurement study across several representative smart glasses: **Ray-Ban (RB)-Meta G1** [47], **RB-Meta Display** [46], **Even G1** [59], and **Cyan** [65]. Across these devices, we study five mobile applications, including live streaming, real-time video conferencing, and conversational AI with voice and video calling, under both controlled experiments and in-the-wild conditions. Most of these glasses are closed platforms that expose little about their internal behavior. However, we were able to obtain SDK access to **Cyan** glasses through developer access¹, enabling us to instrument fine-grained performance breakdowns. We refer to the closed commercial platform and its default application path as **Cyan**, and to our white-box application built using the Cyan SDK as **Dragon**. Dragon complements, rather than replaces, measurements of closed commercial applications.

Beyond software instrumentation, characterizing power consumption on closed-form wearables presents a challenge since internal power rails are inaccessible and battery-based measurements are noisy and unreliable. We address this by tearing down the glass exterior, exposing charging leads and connecting them to a controlled external power supply and shunt resistor. Current is measured by the external supply/shunt setup and voltage by a digital multimeter; GPT-4o is used only to digitize displayed readings from recorded frames.

Key systems contribution. Across all the glasses and applications, we measure latency end-to-end, across glass, phone, and the cloud pipeline, and uplink/downlink throughput, video quality, and power consumption, etc. To obtain these metrics, we develop a suite of instrumentation tools—packet sniffers on Bluetooth and Wi-Fi Direct links, application-level logging on phones, and synchronized timestamping across all devices, including a custom glass prototype for controlled experiments. We then compare and contrast how the same application behaves on different glasses, and also how the same glasses behave when the application is accessed directly on a smartphone vs. the glass. We summarize our key findings below:

- **Smartphones transcode glass video before upload:** Live streaming from **RB-Meta G1** on Instagram and Facebook triggers a costly “decode–upscale–re-encode” pipeline on the companion phone. Contrary to the conventional belief that the constrained G-UL Bluetooth link is the bottleneck, we find that the dominant source of latency is the phone’s transcoding pipeline – the glasses stream a low-resolution video to the phone, which decodes, upscales, and re-encodes it before upload, spiking CPU utilization to 250–350% compared to 100–200% for phone-native streaming. This transcoding overhead, not the wireless link, is what drives the 2 – 3× latency gap between glass-based and phone-native live streaming (§4.1).
- **Video conferencing from glasses reveals a fundamental upstream and downstream asymmetry on phones:** Video conferencing on glasses with WhatsApp and Messenger reveals a fundamental asymmetry in how **RB-Meta Display** glasses handle downstream and upstream video. On the *display side*, although the smartphone receives a high-res video stream, it decodes, downscales, and crops the frames to 360×640 before forwarding them to the glasses, resulting in a much lower bitrate (~340 kbps) on the G-DL. This aggressive downscaling is due to small 20° FoV of the glass. In contrast, on the *uplink* path, the companion phone relays the video to servers without additional processing, unlike the live-streaming pipeline, to have low latency interaction (§4.2).
- **Live AI sacrifices video fidelity to sustain continuous multimodal interaction:** Meta Live AI interaction streams video at extremely low fidelity, roughly 10× lower bitrate than regular live streaming from glasses. Even though Meta Live AI continuously captures and streams video to the server for multimodal reasoning, the system transmits only a low-res RTP feed at 150–200 kbps, which is an order of magnitude lower than the 2–3 Mbps rates used by live streaming from glasses (§4.3).
- **None of the glasses meets the responsiveness bar for real-time AI interaction:** Across AI voice and Live AI interactions, every measured platform exhibits more than 1 s of end-to-end latency, well above the sub-300 ms responsiveness commonly associated with interactive communication [15, 55]. Major root causes across workloads include G-UL transfer, speech-to-text (STT) and voice activity detection (VAD).
- **Glass design choices drive a 10× latency disparity across devices:** Among all the glasses studied, **RB-Meta G1** consistently shows the lowest latencies while **Cyan** the highest. For example, **RB-Meta G1** exhibits a median latency of 1 second for voice interaction while **Cyan** exhibits around 10 seconds (§4.4). Similarly, **RB-Meta G1**

¹The Cyan SDK was obtained for research purposes only and the company has no involvement in this study.

transmits less data for the same application and interaction compared to all other glasses, suggesting **RB-Meta**'s highly optimized native pipelines for voice and video.

- **Cross-cutting factors:** Glass to phone distance and physical obstructions (e.g., walls) play a key role in shaping application performance, and different glasses respond to wireless degradation in markedly different ways. **RB-Meta G1** maintains stable AI latency with no observable degradation even at long ranges (up to ~50 m in open space), after which the link drops. In contrast, **Cyan** shows a gradual increase by 5 seconds around 50 m driven by packet loss and retransmissions, and disconnects afterwards. Indoors, where walls attenuate signals more sharply, all glasses sustain near-identical latency to their close-range performance but disconnect much earlier (e.g., within 10–15 m depending on layout). Supply-side and battery trends are substantially higher for workloads involving sustained or bursty G-UL transfer than for local capture under the same setup. Continuous video uplink (e.g., live streaming) nearly doubles the measured voltage drop compared to local camera capture, and AI workloads exhibit sharp workload-level power spikes when image transfer occurs.

All data, scripts, and application code used in this study are publicly available [4].

Implications for mobile systems research. Our study surfaces a new class of mobile systems challenges that the community has not looked yet. In many ways, today's glasses resemble IoT devices in their resource constraints and reliance on companion devices and wireless connectivity, but introduce substantially richer sensing, computation, and interactive workloads. This opens rich research directions such as rethinking short-range radio protocols for energy-constrained wearables, redesigning how smartphones serve as edge servers for glasses, developing QoE models that account for multi-hop glass-to-phone-to-cloud pipelines, and building adaptive systems that jointly optimize wireless transmission and on-phone compute. As glasses evolve from niche accessories toward the next mainstream mobile platform, we believe this work provides the community with a necessary foundation for that effort.

2 Background

The evolution of XR and AI glasses has followed two distinct trajectories. The first, dominated by head-mounted displays such as Meta Quest [48], Apple Vision Pro [10], Microsoft HoloLens [51] etc, whose main focus was delivering high quality immersive experiences. While powerful, these devices remain bulky and heavy, limiting their suitability for prolonged use. The second trajectory emphasizes reducing weight and form factor by shifting computation elsewhere. Devices like Magic Leap [39] or XReal [3], while slimmer

than full headsets, depend on a tethered computer connected over a cable for rendering computation and Internet connectivity. More recent devices like **RB-Meta G1** [1], Rokid [2], Brilliant Labs Frame [12], **Even G1** [59], and others have emerged, envisioning all-day wearable, free style glasses that provide continuous connectivity and intelligence without tethering. These devices aspire to combine comfort, style, and utility, and become as ubiquitous as today's smartphones.

Glass connectivity via companion devices. To achieve their lightweight and socially acceptable form factor, these glasses must operate under severe constraints on battery capacity, on-board compute, and thermal dissipation. It is infeasible for these small devices to run heavy AI models or manage high-bandwidth data transmission independently. Consequently, most glasses ship with a companion device (e.g., a compute puck or phone) for storage and connectivity. All Internet interactions happen via the companion device.

Use cases and capabilities. XR and AI glasses support a wide range of Internet applications by combining lightweight sensing and intelligent assistance. In Internet consumer settings, popular use cases include AI voice interaction [17, 61], image recognition [14, 29], real-time language translation [53], and live video streaming from glasses [6]. Users can also receive video playback (e.g., WhatsApp video call) and chat messages on the glasses' display [46], and navigate with map directions. Commercial examples, such as **RB-Meta G1** [56] and **Cyan** glasses [65], handle standard AI voice interactions by streaming speech to a companion device, which relays data to cloud services. **RB-Meta G1**'s 'Live AI' [49] sends continuous camera feed for server-side analysis when prompted. **RB-Meta G1** also supports live streaming to platforms like Insta or FB Live. Simpler models like the **Even G1** [59] display only text display. More recent **RB-Meta Display** glass [46] has full color display and can render video calls received from the WhatsApp and Messenger platforms. Figure 1 shows an example of how a WhatsApp video is rendered on the glass.

3 Measurement Setup and Methodology

3.1 Glass Platforms

Conducting an in-depth measurement study of XR/AI glasses presents several challenges, primarily due to the closed nature of most commercial platforms. Most glasses expose only high-level functionality and reveal little about their system design, protocol behavior, or proprietary firmware. As a result, we are restricted to indirect measurement tools, e.g., Wireshark [66] packet captures or OS-level logs, which limits the granularity and interpretability of performance results. For closed platforms, we combine repeatable packet traces, Bluetooth/Wi-Fi traces, OS logs, companion-phone

Table 1: Device and application coverage. “Yes” denotes an evaluated workload; “–” indicates that the workload is unavailable or does not expose a comparable cross-device implementation. Coverage varies because current smart glasses differ in sensing/display capabilities and in the applications exposed by each platform. AI voice is the only workload broadly supported across all platforms. Dragon is our white-box application built on the Cyan SDK.

Platform	Role	Live stream	Video conf.	Live AI	AI voice	Voice-image AI
RB-Meta G1	Closed commercial	Yes	Yes	Yes	Yes	Yes
RB-Meta Display	Closed commercial	–	Yes	Yes	Yes	Yes
Even G1	Closed commercial	–	–	–	Yes	–
Cyan	Closed commercial	–	–	–	Yes	Yes
Dragon	White-box SDK app	–	–	–	Yes	Yes

traces, synchronized latency measurements, and application-observed events. We report inferred mechanisms as platform-specific when direct internal instrumentation is unavailable.

To dissect performance behaviors further, we obtained SDK access to the **Cyan** glasses, which provides access to sensing, networking, and application-layer APIs. This enables us to implement custom applications and log detailed performance metrics. Cyan denotes the closed commercial platform and default software path; Dragon denotes our white-box application built on the same platform using the Cyan SDK, not a separate commercial glass. For example, we can adjust the specific type of voice, speech, or language AI model being used in order to check how performance changes with each model. We can also log timestamps and bytes accumulated within the app to obtain more robust and accurate measurements.

We exclude traditional XR headsets, such as Apple Vision Pro [10], Microsoft HoloLens [51], and Meta Quest 3 [48], as these devices operate as standalone platforms, and are not designed for all-day wearable use. We also exclude audio-only glasses (e.g., Bose Frames [18] and Amazon Echo Frames [8]) that provide basic services such as music streaming but lack multimodal sensing or AI-driven applications.

Table 1 makes the workload coverage explicit. Coverage is determined by two requirements: the platform must provide the necessary hardware capabilities, and its software must expose a supported end-to-end application path comparable to the evaluated workload. Thus, the absence of a workload



Figure 1: Video call on RB-Meta display glass.

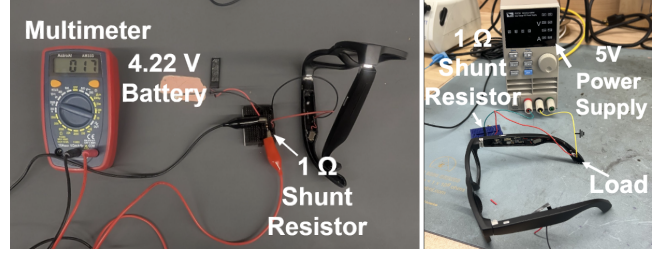


Figure 2: Testbed to measure power: for collecting (a) current, (b) voltage drop.

does not merely indicate that different features were measured; it indicates either missing hardware support or the absence of a comparable application path. Glass-originated live streaming is therefore evaluated on RB-Meta G1, while video conferencing and Live AI are evaluated on the corresponding Meta platforms that expose those workflows. AI voice is evaluated more broadly because it is supported across nearly all platforms. Distance and power experiments are limited to devices that provide both the required workload and compatible instrumentation.

3.2 Applications and QoE Metrics

We study five popular applications that capture the dominant interaction modes: Live Streaming, Video Conferencing, Live AI, AI Voice Interaction, and AI Voice-Image Interaction. Each glass has a subset of these applications, each accessing different sensors, network paths, and cloud pipelines.

- **(1) Live Video Streaming:** This involves continuous transmission of camera frames from the glasses to social media platforms (Facebook, Instagram). For this application, we measure video resolution, latency, and throughput during different phases of a session. We also capture voltage drop and CPU usage of the companion phone.
- **(2) Video Conferencing:** This application enables two-way audiovisual communication directly from the glasses, where the incoming video feed from the remote participant is rendered on the built-in display while the glasses simultaneously transmit the camera feed. For this, we measure bitrate and end-to-end latency. This application complements our other workloads by capturing the behavior of fully interactive, bidirectional video communication initiated and consumed directly on glasses.
- **(3) AI Voice Interaction:** This is a key conversational use case for most glasses. A user speaks a query, the glasses capture audio, and the companion phone forwards the speech to cloud AI models. The response is returned to the phone and relayed back to the glasses as audio. For this application, we measure end-to-end latency, defined as the time from the moment the user finishes speaking to the arrival of the first audio chunk on the glasses.

- **(4) AI Voice-Image Interaction:** This application extends the voice interaction with visual input. When a user asks a question about the scene, the glasses capture and transmit image and audio query to the phone and cloud AI, and receive an audio explanation on the glasses. Here, we measure end-to-end latency from image capture to the first return audio chunk and compute detailed latency breakdowns similar to the voice pipeline.
- **(5) Live AI Interaction:** This is a real-time visual analysis pipeline that integrates AI inference into continuous or quasi-continuous capture of camera streams. For this application, we measure throughput and live latency during the entire conversation.

3.3 Data Collection

We instrument application code, network tracing, and controlled power consumption measurements to capture end-to-end behaviors. We start with **RB-Meta G1** which provides no access to internal instrumentation, so all measurements rely on external tools. For all applications, we captured traffic on the smartphone using Wireshark (Android: rooted Pixel 8; iOS: iPhone 13). We identify uplink and downlink flows between the phone and cloud services, extract end-to-end latency directly from packet timestamps, and compute bitrate patterns. For G-UL and G-DL, Bluetooth traces were obtained via Android HCI snoop logs [9] and Apple's Packet Logger. **Cyan** and **Even G1** are similar in nature to **RB-Meta G1** with no programmability, and we follow a similar procedure to collect data. **Dragon** provides SDK, so we instrumented different applications with different breaking points for fine-grained timestamps. Within this, we probed the AI with 50 different questions in order to test how the capabilities of the model as well as if any meaningful changes in latency occur. These questions can be found in Appendix A. Unless otherwise specified, all experiments are repeated for 15 runs under the same conditions and report average across all runs.

Power measurement methodology and validation: To obtain reliable power measurements, we tear down the glasses by exposing their charging leads and connecting them to a controlled measurement setup. As shown in Figure 2, the glasses were wired in parallel with an external DC power supply (ITECH IT6720 [30]) and a shunt resistor [5]. During each experiment, the supply's current output was recorded by capturing a video of the display. GPT-4o APIs [54] were used only to digitize one displayed current reading per second from the recorded frames; the electrical measurement itself comes from the external supply and shunt. We used this automation because it was more reliable than standard OCR on our recorded display frames, and manually verified the extracted values. We multiply the measured current by the

regulated 5 V to obtain power. The only accessible power signal for **RB-Meta G1** is the battery voltage reported through Android logcat, as the internal power pins are not exposed. To enable a fair comparison with **Dragon**, we reproduced a similar voltage-drop measurement using an external setup: a fixed 4.22 V source placed in parallel with the glasses through a shunt resistor, with voltage across the resistor read from a digital multimeter. As with the current measurements, we recorded the multimeter display and extracted readings. We manually verified the output values reported by the model to validate that they were correctly extracted from the image. These are supply-side, workload-level measurements intended for comparing multi-minute trends under the same setup. They are not rail-level measurements, do not isolate the radio, camera, SoC, PMIC, or display, and do not capture transient current peaks.

Artifacts: Our artifacts include complete packet captures for every device-application pair, companion phone traces, Internet-interaction logs, power and voltage-drop, and the full set of Python scripts used to compute throughput, latency breakdowns, and time-series power profiles. To validate our results, we performed cross-platform comparisons, e.g. Meta on iOS vs. Android, **Cyan** vs. **Dragon**, to ensure that the observed behaviors were consistent across OS, workloads, and measurement tools. The data, scripts, and application code can be found at this link: [4].

4 Characterizing XR and AI Smart-Glasses Application Pipelines

4.1 Live Video Streaming from Glasses

Live video streaming is one of the most demanding workloads for smart glasses, requiring continuous camera capture, video encoding, and sustained wireless transmission under tight compute and energy constraints. Among the platforms in our study, live streaming is supported only by the **RB-Meta** glasses. We focus our evaluation on **RB-Meta G1**, which provides a complete and accessible live-streaming workflow from the glasses to social media platforms; the remaining platforms either lack the required camera capability or do not expose a comparable live-streaming application (Table 1). Live streaming requires sustained high-throughput video upload and timely relaying of encoded frames to social media platforms. Therefore streaming live from glasses raises two questions: 1) *How well can current smart glasses sustain continuous video capture and transmission, and how do device, wireless link, and operating-system differences influence the achievable throughput, video quality, and stability of live streams?* 2) *How do different streaming applications and pipelines determine the end-to-end delay, smartphone workload, and perceived quality of live video originating from glasses?* Figure 6 shows the measurement setup for live streaming.

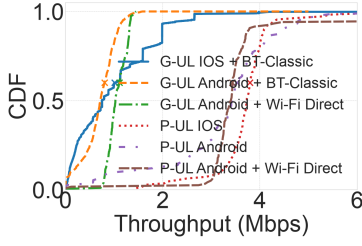


Figure 3: Uplink throughput during Instagram Live streaming from RB-Meta G1 glasses. G-UL throughput is consistently lower than P-UL across all OS and protocol combinations. This is because phone’s video upscaling.

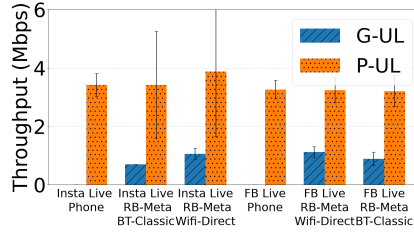


Figure 4: G-UL vs. P-UL throughput across streaming platforms and configurations. Phone-native streaming shows only P-UL throughput since there is no G-UL. In all glass-based cases, P-UL throughput is much higher than G-UL.

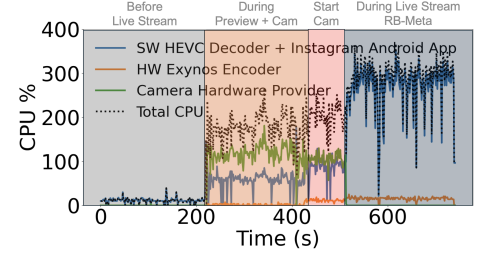


Figure 5: CPU utilization on the companion phone during Instagram Live. Streaming from RB-Meta G1 drives CPU to 250–350%, dominated by the software HEVC decoder and hardware Exynos encoder performing a full decode-encode loop.

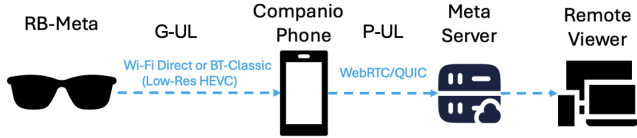


Figure 6: Live video streaming measurement setup: RB-Meta G1 streams video to the companion phone over Wi-Fi Direct or BT-Classic. The phone decodes, up-scales, and re-encodes the video before uploading to Meta’s servers via WebRTC/QUIC.

Figure 3 presents the uplink throughput distribution for live streaming on Instagram from **RB-Meta G1** glasses. The key takeaway here is the difference in throughput between G-UL and P-UL. At first, we thought that the large gap between G-UL and P-UL throughput appears intuitive: the glasses-to-phone link is heavily resource-constrained, often operating over BT Classic or BLE, while the phone-to-server path uses Wi-Fi or cellular, which typically supports much higher bandwidth. However, a deeper inspection through Wireshark tool revealed a surprising mismatch: the total volume of video data, i.e., total bytes sent from the glasses does not align with the significantly larger amount of data transmitted from the phone to the server. The same effect is observed across FB Live as well, and also with BT and Wi-Fi Direct (see Figure 4). This discrepancy prompted a closer examination of the behavior of the companion phone. We verify this with two key observations. First, we observed substantial spikes in CPU utilization during live streaming (compared to when streaming directly from the phone), with utilization reaching 250–350% driven by a software HEVC decoder and hardware encoder running simultaneously. Second, further analysis of Android logcat traces confirmed that the phone is not merely relaying the video, it is actively transcoding the incoming stream, decoding the video received from the glasses and re-encoding it at a higher bitrate

and increased resolution before uploading to Instagram’s servers. We next dive deep into these findings.

(1) Companion phone CPU spikes during glass-based streaming: Figure 5 explains throughput behavior with CPU utilization breakdown on the companion smartphone during an Instagram Live session. Before streaming begins, CPU usage remains low. Once the preview mode using the phone’s own camera is activated, CPU activity increases modestly. However, during actual live streaming from the **RB-Meta G1** glasses, CPU utilization spikes dramatically, reaching 250–350% across multiple cores, driven primarily by two components: the software HEVC decoder, which decodes the incoming video stream from the glasses, and the hardware Exynos encoder, which then re-encodes the video at a higher resolution and bitrate suitable for Instagram’s ingestion pipeline. We find these video transcodings through Android logcat system info which we describe next.

(2) Log traces expose a hidden decode-re-encode loop on the phone: Resolution levels logged on both links confirm this. Across all four app/protocol configurations, over both Bluetooth Classic and Wi-Fi Direct, the glasses send a low, tightly clustered resolution to the phone while the phone uploads a substantially higher resolution to the platform. Appendix C reports the per-configuration distributions (Figure 22) and the resolution-ID mapping (Table 3). These patterns confirm that, the phone is performing a full decode → upscale → re-encode pipeline before transmission. This loop is a measured behavior of the current RB-Meta G1 pipeline, not a universal requirement. The broader systems issue is that a mismatch between glass-native video and application-ingestion formats can turn the companion phone into a hidden media-processing bottleneck.

Additional observations: Figure 3 reveals substantial variation between operating systems and wireless protocols. **RB-Meta**

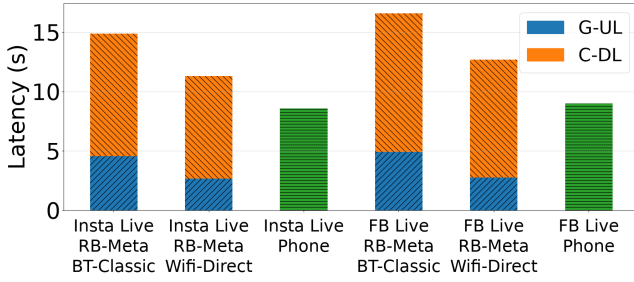


Figure 7: End-to-end latency of live streaming from RB-Meta G1 glasses vs. phone-native (Pixel 8) to remote clients, broken down into G-UL and C-DL (phone-to-client) components. Glass-based streaming incurs 2–3× higher latency than phone-native across both Insta Live and FB Live. G-UL accounts for only ~30% of the total; the dominant source is the phone’s transcoding.

G1 can live stream with both BT and Wi-Fi Direct protocols on Android OS, but it can only support Bluetooth on iOS. By default, **RB-Meta G1** tries to stream via Wi-Fi Direct on Android, but if it fails to connect, it goes back to BT. We can also change the protocol on the Instagram app. Note that despite these differences, *we did not observe any difference in total video bytes sent over G-UL across BT or Wi-Fi Direct.*

Live streaming from glass vs. phone: A key usability question for live streaming is how quickly a viewer actually sees what the glasses are streaming. Unlike throughput or resolution, which affect quality, delay determines whether glasses can support timely, responsive live updates during a broadcast. Figure 7 compares the user-perceived end-to-end latency when streaming from **RB-Meta G1** glasses versus directly from the phone. The end-to-end latency includes G-UL latency and phone to client i.e., remote user device that is co-located in the same LAN (we denote this as C-DL latency). Glass-based streaming experiences roughly 2–3× higher latency than phone-native streaming on both Insta Live and FB Live. At first glance, this gap appears attributable to a slow G-UL since the glasses are resource-constrained, which we initially assumed to be the primary cause of overall latency. However, our analysis found that G-UL accounts for only 30% of total latency. The dominant source of latency is the C-DL stage, which includes the phone’s full transcoding pipeline: decoding the video received from the glasses, upscaling it, and re-encoding it at a higher resolution before uploading it to the platform.

Takeaway: Live-streaming latency is not determined primarily by the constrained G-UL. In the measured RB-Meta G1 pipeline, phone-side decoding, upscaling, and re-encoding dominate the latency gap, showing that representation mismatches can turn the companion phone into a hidden media-processing bottleneck.

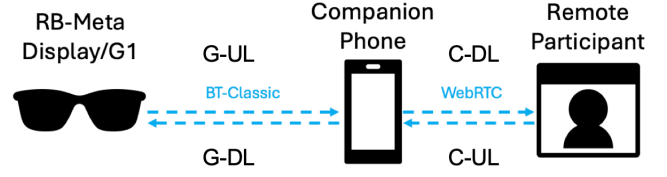


Figure 8: Video Conferencing measurement setup: the remote clients send video via WebRTC to glass via the companion phone. Glasses stream first-person view to the remote clients.

4.2 Video Conferencing on Glasses

Video conferencing represents the most demanding end-to-end workload supported by today’s consumer smart glasses. This subsection is scoped to **RB-Meta Display**, which exposes two-way conferencing with an integrated display in our study. With the **RB-Meta Display** glasses, users can not only stream live video from the glasses to another participant, but also receive an incoming video feed and rendered directly onto the glass display (Figure 8). Both directions of traffic traverse the phone, which handles encoding, decoding, and relaying for every frame exchanged between the glasses and the remote peer. This raises two questions: (1) *How well can current smart-glasses platforms sustain real-time, two-way video interaction?* (2) *What system bottlenecks dominate this performance, and how do companion-phone processing stages reshape latency and bandwidth in each direction?*

We experiment video conferencing on **RB-Meta Display**, the first consumer smart-glasses platform to support true two-way video calls with an integrated display, making it crucial to understand how such a capability stresses the system. Unlike live streaming, video conferencing requires tight timing to maintain real-time interaction, consistent framerate delivery, and symmetric bandwidth in both directions. These requirements have direct implications for throughput, latency, battery consumption, and thermal behavior. We characterize these effects in detail, beginning with the throughput and latency dynamics observed during WhatsApp and Messenger video calls.

Figure 9 shows perceived video-conferencing latency on the **RB-Meta Display** glasses in two directions: (a) when a remote client streams video to the glasses, and (b) when the glasses stream video to a remote client. Across both Messenger and WhatsApp, we observe media end-to-end display latencies often reaching ~700 ms with notably high variance when streaming *to* the glasses, and 250 ms when streaming *from* the glasses. Even with all devices on the same LAN, these latencies substantially exceed the conventional (e.g., Zoom, FaceTime, and regular WhatsApp calls) 200–300 ms range associated with good video-conferencing QoE. The key reason for this discrepancy is the companion phone’s processing pipeline. When streaming *to* the glasses, the remote client sends a high-resolution (e.g., 720p) stream to

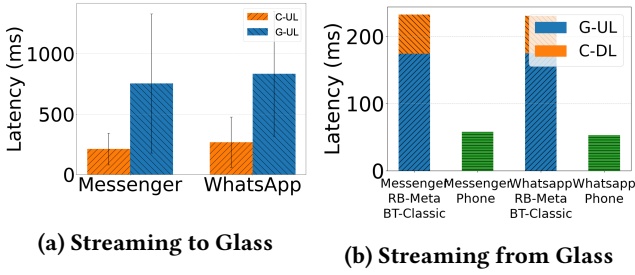


Figure 9: Perceived video-conferencing latency on RB-Meta Display glasses shows two patterns: (a) C-UL and G-DL, which together form client-to-glasses link, when a remote client streams video to the glasses, and (b) G-UL and C-DL, which together form glasses-to-client link, when the glasses stream video to a remote client. Even with the glasses, companion phone, and remote device all on the same LAN, the end-to-end display latency is disproportionately high and far exceeds the conventional 200–300 ms range typically associated with good video-conferencing QoE.

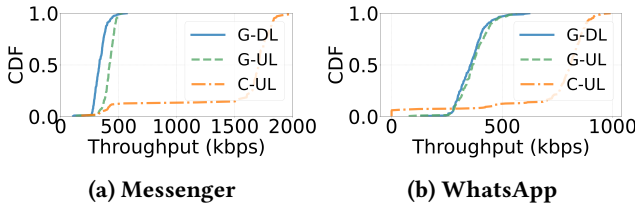


Figure 10: Video-conferencing throughput distributions on RB-Meta Display for (a) Messenger and (b) WhatsApp. Here, G-DL and G-UL denote phone-to-glasses and glasses-to-phone traffic, respectively, while C-UL denotes remote-client-to-phone traffic.

the companion phone (which is displayed on the phone). However, phones decode, crop, and re-encode the video into the glasses' much lower-resolution, narrow-FoV (600×600, ~20°) display format. This transcoding and spatial cropping pipeline introduces substantial delay, and its latency varies significantly depending on the incoming video resolution i.e., higher resolution input results in longer decode-re-encode times and vice versa, contributing to the large variance observed. In contrast, when streaming *from* the glasses, the companion phone performs no transcoding, instead relaying the captured stream directly to the conferencing app to preserve low latency.

We observe similar asymmetry for throughput, consistent with the latency pattern. Figure 10 shows the throughput distribution across the G-DL, G-UL, and C-UL for Messenger and WhatsApp video calls to and from the glasses. The C-UL (remote client to companion phone) throughput is the highest, often reaching 1 Mbps, with high-resolution (e.g., 720p)

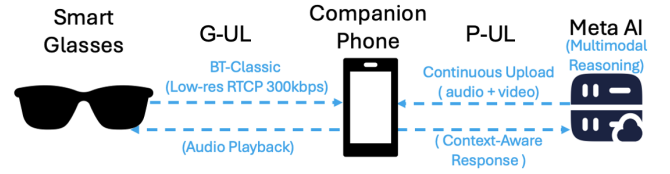


Figure 11: Live AI Interaction measurement setup: glasses continuously stream low-res RTP video and audio to the phone, which uploads to Meta AI for ongoing multimodal reasoning. Context-aware audio responses are returned in real time.

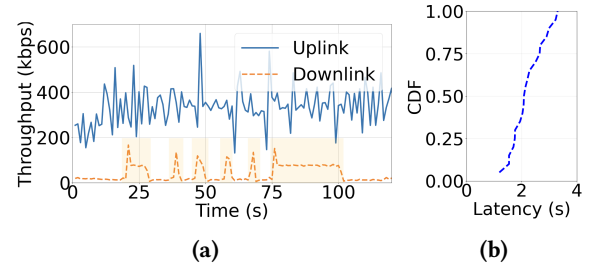


Figure 12: Live AI on RB-Meta glasses. (a) Uplink and downlink throughput over a continuous session. (b) CDF of per-update latency

video received by the companion phone. In contrast, the G-DL (phone to glasses) throughput is substantially lower because the companion phone downscales and crops the incoming video to the glasses' narrow-FoV, 600×600 display format. From our measurements, we observe that the phone downscales and crops the frames to 360×640 despite having higher resolution display. On the other hand, the G-UL and C-UL throughputs closely match when streaming *from* the glasses, since the companion phone performs no upscaling and simply relays the captured frames directly to the conferencing application.

Takeaway: Video conferencing avoids upstream transcoding but aggressively downscales downstream video for the glass display. The broader lesson is that upstream and downstream glass video have different latency, bitrate, and processing requirements and should be optimized independently.

4.3 Live AI Interaction

Live AI on Meta glasses exposes a new interaction mode where the AI assistant stays continuously engaged with the user and their surroundings, rather than responding to isolated queries. Once activated by user (e.g., user says "Hey Meta, start Live AI", after that the user doesn't need to say "Hey Meta" for the rest of the session.), the glasses keep the camera and microphone running so that Meta AI can generate faster, context-aware responses tailored to whatever the user is currently seeing or saying. Instead of issuing a single request and waiting for a one-time answer, the user

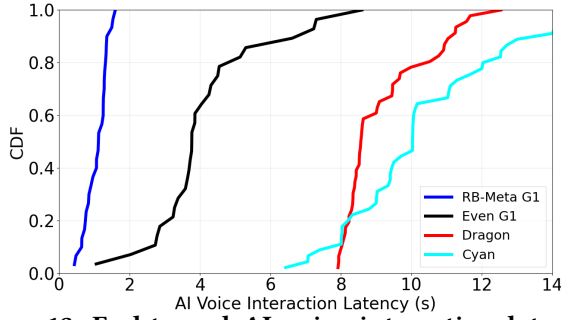


Figure 13: End-to-end AI voice interaction latency across all the tested glasses. RB-Meta G1 exhibits the fastest response times, while Cyan shows the highest.

receives a steady stream of updated interpretations as they move, look around, or ask follow-up questions. This makes Live AI fundamentally distinct from the other applications in our study with very different traffic characteristics (Figure 11). Therefore, our goal in this study is to answer: *How does continuous, always-on multimodal sensing in Live AI reshape the end-to-end latency, traffic patterns, and resource usage compared to traditional one-shot AI interactions?*

To understand how Live AI behaves under continuous sensing, we measure both the throughput and per-update latency of its multimodal pipeline. Figure 12a shows the uplink and downlink throughput patterns. Although Live AI keeps both the camera and microphone continuously active, the resulting data transfer is surprisingly lightweight. Across the full session, the average uplink throughput hovers around only 300 kbps, over an order of magnitude lower than live video streaming and several times lower than video conferencing. This aligns with the design of Live AI: unlike human-facing video workloads like live streaming and conferencing, it does not require high-resolution or high-frame-rate video. Instead, the glasses transmit only small, low FPS video sufficient for the AI model to interpret the scene. Our logcat traces confirm this behavior, showing that the glasses typically send fewer than 10 frames per second, and that frame sizes vary considerably depending on lighting, motion, and scene content. These mixed, variable-size frames explain why the uplink time series in Figure 12a lacks distinct spikes for individual voice queries, as the video and audio streams are interleaved into a single fluctuating uplink pattern.

Given this lightweight but continuous multimodal upload pattern, the next question is how quickly each Live AI update returns to the user, i.e., how its end-to-end latency compares to the other AI interactions. As Figure 12b shows, Live AI’s per-update latency is noticeably higher than voice-only interaction, despite continuous capture and the frequent contextual updates provided to the AI models.

Takeaway: Live AI substantially reduces machine-viewed video fidelity, but its response latency remains high despite

the low video bitrate. This indicates that reducing continuous video bitrate alone is insufficient; cloud inference and response generation remain important latency constraints.

4.4 AI Voice and Image Interaction

Two fundamental AI interactions that define the everyday user experience in smart glasses today: AI Voice and Voice-Image. Glasses support these interactions purely with voice and also describe environment with an image (Figure 15). Our goal in this study is to answer two key questions: 1) *How quickly and reliably can current glasses produce spoken AI responses to user queries, and which components of the end-to-end processing chain contribute most to latency?* 2) *How do different device design choices translate into measurable differences in latency, throughput, packet dynamics, power consumption, and overall interaction behavior across workloads?*

To ensure consistency across devices, we unify our analysis of AI Voice and Voice-Image interactions as follows. Both Meta glasses (G1 and Display) exhibit nearly identical behavior for both workloads, so for brevity, we report **RB-Meta G1**’s performance in this section. For **Dragon**, we implement a fully custom client using either Deepgram [23] voice agent for voice AI or Deepgram [22] speech-to-text and OpenAI [54] speech-generation APIs for voice-image AI (since Deepgram voice agent cannot process images), allowing us to instrument each stage of the pipeline. This differs from **Cyan**, which uses text-to-speech (TTS) and speech-to-text (STT) models from Microsoft Azure. Besides the use of these APIs, **Dragon** makes the optimization of using the Bluetooth connection of the glasses to intake audio, while **Cyan** downloads an opus file and streams it to the TTS/STT server. Similarly to **Cyan**, **Even-G1** uses the official companion app obtained from the app store, preserving their default interaction flow.

Figure 13 shows the end-to-end latency of AI voice interaction across all glasses tested within 46 different tests. **RB-Meta G1** consistently delivers the fastest responses, containing a median latency of 1.12 seconds. This reflects both the efficient audio capture and aggressive AI cloud processing of **RB-Meta G1**. **Even-G1** follows next, with a median latency of 3.77 seconds. **Dragon** exhibits a further shift, with a median latency of 8.57 seconds. **Cyan** shows the slowest performance, with a median latency of 10.03 seconds.

Breakdown of the latency: To dissect the end-to-end latency of AI voice interaction, we evaluate two major sources of variability: the impact of different AI model choices and the contribution of individual pipeline components.

Figure 14 shows this analysis. Across 30 tests of various LLMs, Figure 14a displays clear differences in end-to-end latency: we see that Claude Sonnet contains the highest latency by far, followed by Claude Haiku, Gemini, GPT-4o, and

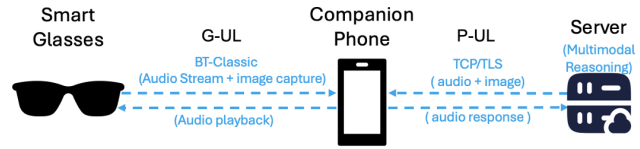
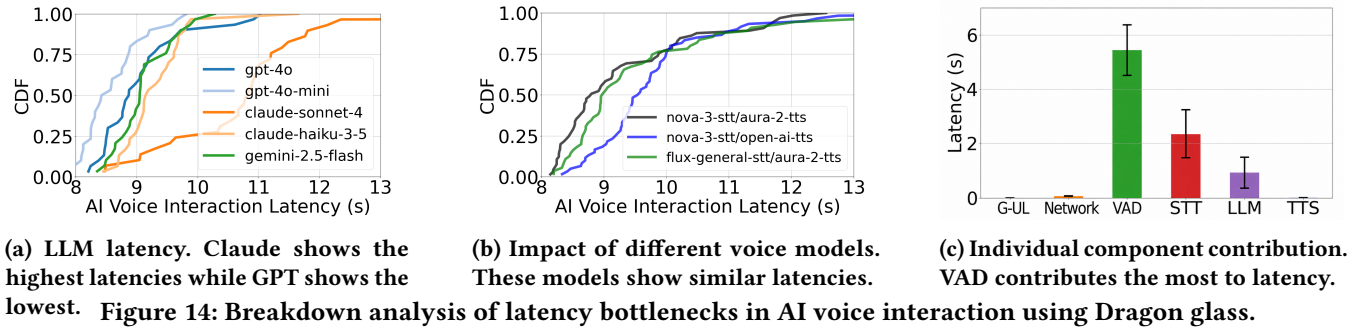


Figure 15: AI Voice-Image Interaction measurement setup: same as voice AI, but the glasses additionally capture and transmit an image over G-UL. The image must be fully delivered before the multimodal LLM can begin reasoning.

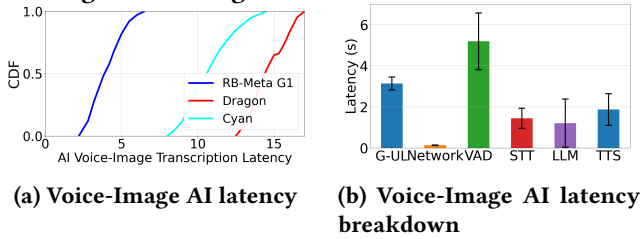


Figure 16: Voice-Image AI latency across RB-Meta G1, Cyan, and Dragon, with the Dragon breakdown separating G-UL image transfer, cloud processing, and return-audio stages.

lightweight GPT-4o-mini, which is consistently the fastest. These variations arise from larger models taking up more inference time and producing more verbose responses, which increase downstream processing and audio generation time.

In contrast, Figure 14b shows that STT/TTS choices introduce only modest differences. Most configurations yield similar latencies because TTS contributes only the time needed to produce the first audio chunk. Figure 14c displays the breakdown of individual latencies when testing Voice AI. Within this, the voice activity detection (VAD) latency (*defined as the time between sending the first speech chunk to audio being recognized in the server*) is the main bottleneck, taking up a median of 5.35 seconds.

Voice vs. Voice-Image AI interaction: Injecting an image into the AI pipeline adds an additional sensing and processing step, reshaping the latency profile in a measurable way. Figure 16 shows that each device experiences an additional

1 to 3 second delay when image processing is included. In order, **RB-Meta G1** has a median latency of 3.45 seconds, followed by **Cyan** 11.77 seconds, and **Dragon** 14.65 seconds.

Figure 16b decomposes these delays and highlights the key structural shift relative to voice-only interaction: *the G-UL stage becomes a major bottleneck*. For voice-only queries, the G-UL contributes almost no latency because all glasses use highly optimized BT-specific audio drivers that stream small packets while the user is still speaking. For voice-image queries, however, the system must capture a full-resolution image and transmit it to the phone before processing can begin. As a result, **RB-Meta G1** incurs a delay of roughly 1 second, while **Dragon** requires a median of 3.18 seconds for G-UL transmission. This is also because unlike voice, the image cannot be incrementally streamed and must be fully delivered before the multimodal LLM can start reasoning. In **Dragon**, bytes from the image are streamed via BLE in batches, which are then concatenated and sent to the LLM, incurring more processing time.

Uplink and downlink behavior of AI interactions: A key step in understanding AI interaction performance is examining how data flows across the glasses → phone → cloud pipeline. Figure 17 illustrates the uplink and downlink bitrate patterns for **Dragon** and **RB-Meta G1** during voice-only and voice-image interactions. Across all four cases, we observe: (i) an initial G-UL phase where the glasses stream audio (and an image) to the phone, (ii) a P-UL phase where the phone sends the request to the cloud and waits for the AI model to generate a response, and (iii) a P-DL/G-DL phase where the audio reply is delivered back to the glasses. For **RB-Meta G1** voice-only AI shown in Figure 17a, we first see a period of silence, before the user states "Hey Meta" to start the voice AI process, where a small uplink bitrate is recorded. The G-UL segment is short and low bitrate because audio is streamed in small BLE packets while the user is still speaking. This results in modest uplink activity followed by a slightly larger downlink during P-DL. For voice-image AI displayed in 17b, we see another initial period of silence before the user states "Hey Meta". Once the user starts speaking, the audio data is sent over along with the response and the server waits.

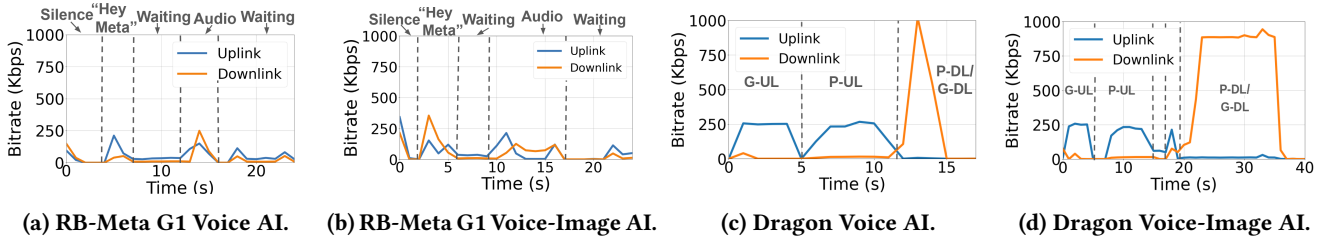


Figure 17: Throughput for Voice AI and Voice-Image AI. The traces identify the initial G-UL upload, the phone-to-cloud processing interval, and the P-DL/G-DL audio-response phase; Dragon uses larger and longer bursts.

When the user asks a question about their surroundings during G-UL, the uplink has a bit larger of a bitrate due to the server having to process an image and the downlink is a bit longer due to the voice response being also longer.

In Figure 17c, the throughput of voice AI is shown in **Dragon** glasses. **Dragon** exhibits a larger, longer uplink burst than **RB-Meta G1**, consistent with its higher G-UL latency. Once the request reaches the cloud, the P-UL region appears as a mostly idle window-latency dominated by server-side STT/LLM/TTS processing with minimal network transfer. After this, the P-DL/G-DL phase produces a pronounced downlink spike corresponding to the synthesized audio response being streamed back to the glasses. **RB-Meta G1**'s downlink bursts are shorter and more compact than **Dragon**'s, showing both its tighter cloud integration and faster end-to-end pipeline. In Figure 17d, the throughput of voice-image AI is shown in **Dragon** glasses. Within this, we see similar G-UL/P-UL ranges as voice AI.

Takeaway: AI latency accumulates across sensing, transfer, endpointing, cloud reasoning, and response delivery rather than at a single link. Streaming audio can overlap G-UL transfer with speech.

5 Cross-Cutting Factors

Beyond individual application workloads, we observe two fundamental factors that shape how smart glasses perform in everyday settings: distance between the glass and phone, and power consumption under different interaction modes. In this section, we quantify how distance and power jointly and/or individually affect G-UL and G-DL performance.

5.1 Impact of Distance between Glass and Phone

Across all glasses, we find that distance has far less impact on perceived latency than expected. Up to 50 meters, every glass except **Cyan** maintains nearly the same end-to-end latency for both voice and voice-image AI interactions. At 50 meters, **Cyan** incurs an additional latency of up to 5 seconds, while the other glasses remain within their baseline

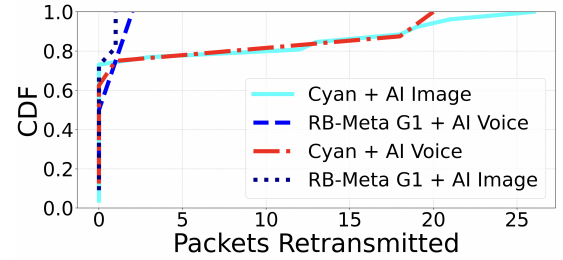


Figure 18: TCP packet retransmissions at 50 m G-UL distance for AI interactions. Cyan exhibits 15–25 retransmissions during AI image queries. This helps explain its multi-second latency inflation at long range. **RB-Meta G1** remains stable with almost no retransmissions across both workloads. At shorter distances (0–40 m), neither glass exhibited retransmissions.

ranges. Beyond 50 meters, all glasses begin to exhibit repeated disconnects, unstable link recovery, and unusable interaction performance, which are consistent with widely accepted BT range limits. All glasses transmit AI interaction data over TCP-based P-UL, which allows us to directly log packet retransmissions during long-range operation.

To understand **Cyan**'s latency inflation, we examine G-UL reliability packet losses and retransmissions. Figure 18 shows that **Cyan** experiences a large packet retransmission burst (15–25) during AI image queries at 50 meters, while **RB-Meta G1** records almost no retransmissions and remains stable across workloads. This spike in retransmissions directly explains **Cyan**'s high delay increase at long range.

Takeaway: Distance alone does not gradually increase latency for most devices; performance remains flat until the link approaches failure. Near that boundary, platform-specific recovery behavior, rather than distance itself, determines whether latency inflates or the connection simply drops.

5.2 Power Draw Across Glasses and Applications

We now examine how different interaction modes influence device-level power consumption. Power behavior reflects both the intensity of sensor activity (microphones, cameras), and the volume and timing of G-UL transmissions.

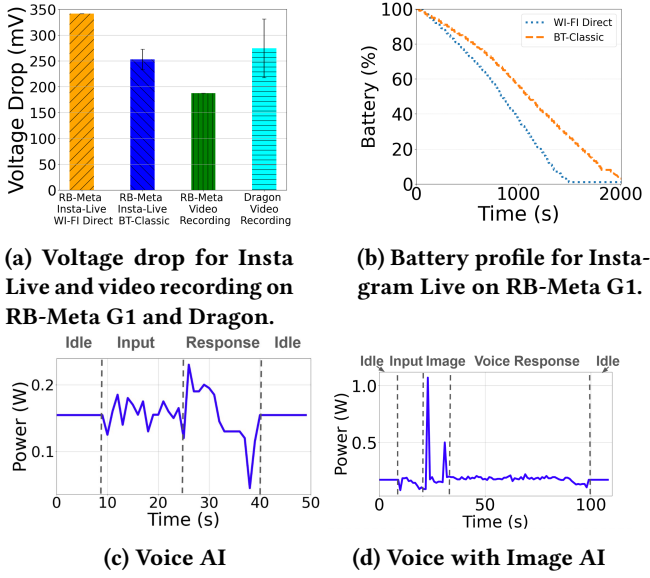


Figure 19: Power consumption across applications.

Figure 19a reports the voltage drop across different operations on **RB-Meta G1** and **Dragon** glasses over a 3 minute duration. We observe that Insta Live on **RB-Meta G1** using Wi-Fi Direct incurs the highest voltage drop (~ 340 mV), reflecting the sustained high-throughput video uplink and continuous HEVC encoding on the glasses. Switching to BT-Classic reduces the drop to ~ 250 mV, but still represents a substantial power draw due to the need to maintain a steady video stream over a low-bandwidth wireless link. Dissecting this based on local video recording on **RB-Meta G1** shows a significantly lower drop (~ 180 mV), suggesting that camera capture and video encoding forms the baseline (~ 180 mV), and Wi-Fi Direct G-UL streaming almost doubles this cost (~ 340 mV). This indicates that the radio and uplink processing add a power overhead comparable in magnitude to the camera itself. **Dragon** exhibits a higher drop (~ 280 mV) even for local recording, with noticeably larger variance. Figure 19b compares how quickly glasses drain when running a high-demand live video streaming application on **RB-Meta G1**: Insta Live over Wi-Fi Direct and Insta Live over BT. Both Insta Live modes deplete the battery and stop in roughly 25 to 35 minutes for Wi-Fi Direct and about 40 minutes for BT-Classic. Wi-Fi Direct drains slightly faster because it maintains a higher-power glasses radio uplink. We also measure video conferencing applications

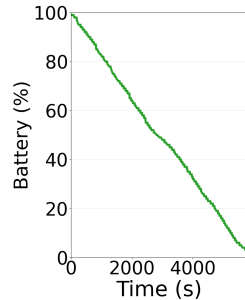


Figure 20: Battery drain during a Messenger video call on RB-Meta Display.

on **RB-Meta display** glass. Figure 20 shows that the Messenger call on **RB-Meta display** glass lasts almost 80 minutes. Although this is longer than we expected, we note that this is an intuitive result because this glass has more battery capacity (248 mAh) than the **RB-Meta G1** glass (154 mAh).

While the above voltage-drop and battery measurements provide a coarse indication of energy usage, they are insufficient to reveal the fine-grained power dynamics. To obtain a more direct view, we instrument the **Dragon** prototype by exposing its internal power rails (as described in §3), studying its AI workload since it lacks support for live video streaming. Figure 19c shows that the power consumption of voice AI has a moderate oscillation between ~ 0.14 and ~ 0.18 W during the user’s speech input, corresponding to the G-UL of audio via BT. Once the voice response is generated in the cloud and streamed back to the glasses, power rises to ~ 0.22 W before gradually returning to the idle baseline of ~ 0.16 W. For Voice-Image AI (Figure 19d), the initial voice-input segment mirrors Voice AI, but the moment the user triggers an image query, we observe a sharp power spike exceeding 1 W as the glasses capture and transmit an image. This is followed by a second, smaller rise when the generated response is delivered back to the device, after which power settles into a mid-level oscillation.

Takeaway: Sustained video transfer coincide with higher energy cost. This motivates reducing transfer volume and duration, while component-level attribution requires finer-grained instrumentation.

6 Discussion

Why Companion-Assisted Architectures Remain Relevant. More onboard compute will not remove the partitioning problem [38, 60, 68]. All-day glasses remain constrained by weight, battery capacity, thermal dissipation, antenna placement, and socially acceptable form factor [41, 64]. A phone or nearby edge device can remain useful for expensive computation, networking, storage, and cloud coordination [26, 37, 42]. Even more standalone glasses must balance sensing fidelity, local compute, wireless transfer, cloud latency, power, and heat [43, 45]. Our results should be interpreted as evidence about where work is placed across the glass, companion, and cloud.

From Implementation Choices to Design Directions. Table 2 separates measured observations from inferred platform behavior and the architectural principles they suggest; because these platforms are closed, implementation details come from external traces [9, 66] and are not universal requirements. Two patterns recur: mismatched glass and application formats push media processing onto the phone, as with transcoding-dominated live streaming on **RB-Meta G1**, and conferencing adapts downstream video for the glass display while upstream video prioritizes interaction latency [20].

Table 2: Findings, inferred platform behavior, and architectural principles.

Finding	Observation	Platform behavior	Architectural principle
Live streaming (§4.1)	Phone CPU 250–350%; G-UL only ~30% of latency	App re-encodes video for Instagram and FB Live profiles	Format mismatch makes the phone the bottleneck
Conferencing (§4.2)	G-DL 340 kbps vs. 1 Mbps C-UL; 700/250 ms down/up	Crops to the 600×600, ~20° display; no upstream transcode	Up- and downstream video need separate budgets
Live AI (§4.3)	~300 kbps at <10 fps, yet response latency remains high	Capture tuned for the model, not viewing; every update hits the cloud	Lower fidelity saves bandwidth, not latency
AI Voice (§4.4)	1.12–10.03 s median; VAD adds 5.35 s on Dragon	Dragon chains cloud APIs; BLE images batched before submission	Chained stages add delay

Both point to the same directions: treat the phone as an explicit edge server whose buffering and media load is exposed to application control [26, 43]; adopt glass-aware video representations and transcoding controls that avoid unnecessary decode–re-encode loops; reduce G-UL transfer volume through compression, selective upload [21, 42, 45], or privacy-aware local filtering [16, 57]; coordinate endpointing, response length, and progressive delivery [11, 17, 55] in latency-sensitive AI; and tune upstream capture and downstream display adaptation as separate paths.

Scope and Limitations. Closed commercial systems limit internal instrumentation, and some observed behaviors are application- and platform-specific. We mitigate these limitations through cross-device comparisons where supported, repeatable packet and OS-level traces [9, 66], companion-phone measurements, synchronized latency measurements, and Dragon’s white-box instrumentation. Dragon reveals mechanisms only for our SDK application and is not assumed to reproduce Cyan or Meta internals. Device coverage also differs by workload and available power instrumentation, as summarized in Table 1. The power experiments compare supply-side or battery trends under the same setup; they do not provide rail-level attribution, isolate individual hardware components, or capture transient current peaks as in component-level energy characterizations [41]. Results for a specific application should be interpreted as measurements of that platform and workload rather than as representative of all smart glasses. Our cross-platform conclusions are limited to behaviors that recur across multiple devices or are supported by complementary measurements, while platform-specific observations are identified and discussed as such.

7 Related Work

Mobile, wearable and embedded devices. Prior work has extensively characterized the performance of mobile and embedded systems, examining application responsiveness [40, 41, 55, 70] and end-to-end service delays [15]. Wearables such as smartwatches and fitness trackers have been studied for sensor fidelity, interaction latency, and energy constraints under continuous sensing workloads [26, 38].

Recent work in AR/VR focused on local rendering, motion-to-photon latency, and offloading methods for high-quality graphics pipelines [34, 36, 43, 64]. In contrast, our work is the first to measure these effects in several smart glasses.

Sensory AI systems. A large body of work explores cloud- and edge-assisted perception systems [7, 37, 42, 45], speech pipelines [11], and multimodal AI services [27, 33, 72]. Systems such as distributed voice assistants [16, 44] and on-device AI [13, 32, 52, 60, 62, 68] characterize STT, TTS, and large-model processing delays. Unlike prior sensory-compute works, our study exposes how multimodal AI latency is jointly shaped by audio/video interaction, AI models behavior, G-UL and phone-side dynamics, and cloud inference.

Video streaming and conferencing. Video streaming and real-time conferencing have been extensively studied in the context of mobile networks [19, 21, 25, 31, 35, 58, 69, 71]. Recent systems looked at video offload from XR headsets, but assume symmetric throughput and a direct device→server pipeline [20, 43]. Smart glasses break this assumption, relying on constrained short-range links and companion-phone transcoding before video reaches the network.

8 Conclusion

XR and AI smart glasses are rapidly evolving from niche accessories to Internet-connected and all-day wearable sensing platforms. This paper presented the first end-to-end measurement study of this emerging class of devices, covering a variety of glasses across several AI and video applications. Our results show that all glasses incur severe latency across all applications driven primarily by glass to phone wireless links, phone-side processing, and cloud AI pipelines. As the industry pushes toward intelligent, near-eye computing, we believe our findings present a timely opportunity to rethink the wireless, system, and cloud interfaces that will ultimately define the next generation of glass-based computing.

Acknowledgments

This work was partially supported by the U.S. National Science Foundation (NSF) grants IIS-2544317 and TI-2612077.

References

- [1] 2021. Meta Ray-Ban Stories Smart Glasses. <https://www.ray-ban.com/usa/discover-ray-ban-stories>.
- [2] 2023. Rokid AR Glasses and AR Studio Products. <https://www.rokid.com/>.
- [3] 2023. XReal (formerly Nreal) AR Glasses. <https://www.xreal.com/>.
- [4] 2026. <https://github.com/SINRG-Lab/mobicom-26-xr-ai-glass>.
- [5] 2026. Measuring Current using Shunt Resistors — tek.com. <https://www.tek.com/en/blog/measuring-current-using-shunt-resistors>.
- [6] Muhammad Abu-Rmaleh, Tamara Osborn, Santiago R Gonzalez, and James C Yuen. 2022. The use of live streaming technologies in surgery: a review of the literature. *Annals of Plastic Surgery* 88, 1 (2022), 122–127.
- [7] Utku Günay Acer, Marc van den Broeck, Chulhong Min, Mallesham Dasari, and Fahim Kawsar. 2022. The city as a personal assistant: turning urban landmarks into conversational agents for serving hyper local information. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 6, 2 (2022), 1–31.
- [8] Amazon. 2026. Echo Frames (smart glasses with Alexa) — product listing. <https://www.amazon.com/echo-frames/s?k=echo+frames>.
- [9] Charlie Anderson. 2026. How to get the Bluetooth Host Controller Interface logs from a modern Android phone — charlie.d.anderson. <https://medium.com/@charlie.d.anderson/how-to-get-the-bluetooth-host-controller-interface-logs-from-a-modern-android-phone-d23bde00b9fa>.
- [10] Apple Inc. 2024. Apple Vision Pro Augmented Reality Headset. Official product page: <https://www.apple.com/apple-vision-pro/>.
- [11] Mehdi Assefi, Mike Wittie, and Allan Knight. 2015. Impact of network performance on cloud speech recognition. In *2015 24th International Conference on Computer Communication and Networks (ICCCN)*. IEEE, 1–6.
- [12] Brilliant Labs. 2024. Frame: AR Glasses with AI Assistant. Official product page: <https://brilliant.xyz/>.
- [13] Han Cai, Ji Lin, Yujun Lin, Zhijian Liu, Haotian Tang, Hanrui Wang, Ligeng Zhu, and Song Han. 2022. Enable deep learning on mobile devices: Methods, systems, and applications. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 27, 3 (2022), 1–50.
- [14] Kai Chen, Yunhao Gou, Runhui Huang, Zhili Liu, Daxin Tan, Jing Xu, Chunwei Wang, Yi Zhu, Yihan Zeng, Kuo Yang, et al. 2025. Emova: Empowering language models to see, hear and speak with vivid emotions. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 5455–5466.
- [15] Kuan-Ta Chen, Chun-Ying Huang, Polly Huang, and Chin-Laung Lei. 2006. Quantifying skype user satisfaction. *ACM SIGCOMM Computer Communication Review* 36, 4 (2006), 399–410.
- [16] Peng Cheng and Utz Roedig. 2022. Personal voice assistant security and privacy—a survey. *Proc. IEEE* 110, 4 (2022), 476–507.
- [17] Ruizhi Cheng, Surendra Pathak, Guowu Xie, Matteo Varvello, Songqing Chen, and Bo Han. 2025. Hello, GenAI? Dissecting Human to Generative AI Calling. (2025).
- [18] Bose Corporation. 2025. Bose Frames Tenor: Rectangular Bluetooth® Audio Sunglasses. https://global.bose.com/content/consumer_electronics/b2c/north_america/websites/en_ca/product/bose_frames_tenor.html.
- [19] Mallesham Dasari, Arani Bhattacharya, Santiago Vargas, Pranjal Sahu, Aruna Balasubramanian, and Samir R Das. 2020. Streaming 360-degree videos using super-resolution. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 1977–1986.
- [20] Mallesham Dasari, Edward Lu, Michael W Farb, Nuno Pereira, Ivan Liang, and Anthony Rowe. 2023. Scaling VR Video Conferencing. In *2023 IEEE Conference Virtual Reality and 3D User Interfaces (VR)*. IEEE, 648–657.
- [21] Mallesham Dasari, Santiago Vargas, Arani Bhattacharya, Aruna Balasubramanian, and Samir R Das. 2022. Swift: Adaptive video streaming with layered neural codecs. In *NSDI*. 103–118.
- [22] Deepgram, Inc. 2025. Deepgram Speech-to-Text API Documentation. <https://developers.deepgram.com/>.
- [23] Deepgram, Inc. 2025. Deepgram Voice Agent API Documentation. <https://developers.deepgram.com/docs/voice-agent>.
- [24] Oscar Falmer. 2025. Smartglasses Complete Guide. <https://docs.google.com/spreadsheets/d/1zTOeNmBPijGuqm99tdBJhV-hE5NU74sd55H3Fmbf5v4>.
- [25] Himanshu Gupta, Max Curran, Jon Longtin, Torin Rockwell, Kai Zheng, and Mallesham Dasari. 2022. Cyclops: An FSO-based Wireless Link for VR Headsets. In *Proceedings of ACM SIGCOMM Conference*. 601–614.
- [26] Kiryong Ha, Zhuo Chen, Wenlu Hu, Wolfgang Richter, Padmanabhan Pillai, and Mahadev Satyanarayanan. 2014. Towards wearable cognitive assistance. In *Proceedings of the 12th annual international conference on Mobile systems, applications, and services*. 68–81.
- [27] Soyeon Caren Han, Feiqi Cao, Josiah Poon, and Roberto Navigli. 2024. Multimodal large language models and tunings: Vision, language, sensors, audio, and beyond. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 11294–11295.
- [28] David Heaney. 2025. Ray-Ban Meta Glasses Have Sold 2 Million Units, Production To Be Vastly Increased. <https://www.uploadvr.com/ray-ban-meta-glasses-sold-2-units-production-to-be-vastly-increased/>.
- [29] Yushi Hu, Hang Hua, Zhengyuan Yang, Weijia Shi, Noah A Smith, and Jiebo Luo. 2023. Promptcap: Prompt-guided image captioning for vqa with gpt-3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2963–2975.
- [30] Ltd. ITECH Electronic Co. 2025. IT6720 60 V/5 A/100 W DC Bench Power Supply. <https://www.digikey.com/en/products/detail/itech-electronic-co-ltd/IT6720/16685617>.
- [31] Liz Izhikevich, Reese Enghardt, Te-Yuan Huang, and Renata Teixeira. 2024. A global perspective on the past, present, and future of video streaming over starlink. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 8, 3 (2024), 1–22.
- [32] Vijay Janapa Reddi, David Kanter, Peter Mattson, Jared Duke, Thai Nguyen, Ramesh Chukka, Ken Shiring, Koan-Sin Tan, Mark Charlebois, William Chou, et al. 2022. MLPerf mobile inference benchmark: An industry-standard open-source machine learning benchmark for on-device AI. *Proceedings of Machine Learning and Systems* 4 (2022), 352–369.
- [33] Feibo Jiang, Li Dong, Yubo Peng, Kezhi Wang, Kun Yang, Cunhua Pan, and Xiaohu You. 2024. Large AI model empowered multimodal semantic communications. *IEEE Communications Magazine* (2024).
- [34] Tao Jin, Mallesham Dasa, Connor Smith, Kittipat Apicharttrisor, Srinivasan Seshan, and Anthony Rowe. 2024. Meshreduce: Scalable and bandwidth efficient 3D scene capture. In *2024 IEEE Conference Virtual Reality and 3D User Interfaces (VR)*. IEEE, 20–30.
- [35] E Jinlong, Lin He, Zongyi Zhao, Yachen Wang, Gonglong Chen, and Wei Chen. 2024. AggDeliv: Aggregating Multiple Wireless Links for Efficient Mobile Live Video Delivery. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 1173–1180.
- [36] Bill Krogfoss, Jose Duran, Pablo Perez, and Jan Bouwen. 2020. Quantifying the value of 5G and edge cloud on QoE for AR/VR. In *2020 Twelfth International Conference on Quality of Multimedia Experience (QoMEX)*. IEEE, 1–4.
- [37] Guohao Lan, Zida Liu, Yunfan Zhang, Tim Scargill, Jovan Stojkovic, Carlee Joe-Wong, and Maria Gorlatova. 2021. Edge-assisted collaborative image recognition for mobile augmented reality. *ACM Transactions*

- on *Sensor Networks (TOSN)* 18, 1 (2021), 1–31.
- [38] Nicholas D Lane, Sourav Bhattacharya, Petko Georgiev, Claudio Forlivesi, Lei Jiao, Lorena Qendro, and Fahim Kawsar. 2016. Deepx: A software accelerator for low-power deep learning inference on mobile devices. In *2016 15th ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)*. IEEE, 1–12.
 - [39] Magic Leap. 2019. Magic Leap. <https://www.magicleap.com/>.
 - [40] Robert LiKamWa, Bodhi Priyantha, Matthai Philipose, Lin Zhong, and Paramvir Bahl. 2013. Energy characterization and optimization of image sensing toward continuous mobile vision. In *Proceeding of the 11th annual international conference on Mobile systems, applications, and services*. 69–82.
 - [41] Robert LiKamWa, Zhen Wang, Aaron Carroll, Felix Xiaozhu Lin, and Lin Zhong. 2014. Draining our glass: An energy and heat characterization of google glass. In *Proceedings of 5th Asia-Pacific Workshop on Systems*. 1–7.
 - [42] Luyang Liu, Hongyu Li, and Marco Gruteser. 2019. Edge assisted real-time object detection for mobile augmented reality. In *The 25th annual international conference on mobile computing and networking*. 1–16.
 - [43] Edward Lu, Sagar Bharadwaj, Mallesham Dasari, Connor Smith, Srinivasan Seshan, and Anthony Rowe. 2023. Renderfusion: Balancing local and remote rendering for interactive 3D scenes. In *2023 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. IEEE, 312–321.
 - [44] Suresh Malodia, Alberto Ferraris, Mototaka Sakashita, Amandeep Dhir, and Beata Gavurova. 2023. Can Alexa serve customers better? AI-driven voice assistant service interactions. *Journal of Services Marketing* 37, 1 (2023), 25–39.
 - [45] Jiayi Meng, Zhaoning Kong, Qiang Xu, and Y Charlie Hu. 2021. Do larger (more accurate) deep neural network models help in edge-assisted augmented reality?. In *Proceedings of the ACM SIGCOMM 2021 Workshop on Network-Application Integration*. 47–52.
 - [46] Inc. Meta Platforms. 2025. Meta Ray-Ban Display: AI Glasses with In-Lens Display. <https://www.meta.com/ai-glasses/meta-ray-ban-display/>.
 - [47] Inc. Meta Platforms. 2025. Ray-Ban Meta AI Glasses | Meta. <https://www.meta.com/ai-glasses/ray-ban-meta/>.
 - [48] Meta Platforms, Inc. 2023. Meta Quest Virtual Reality Headset. Official product page: <https://www.meta.com/quest/>.
 - [49] Meta Platforms, Inc. 2025. How to use «Live AI» on AI glasses. <https://www.meta.com/help/ai-glasses/894093646030348/>.
 - [50] Meta Platforms, Inc. 2025. Meta Ray-Ban Display AI Glasses & Neural Band. <https://www.meta.com/ai-glasses/meta-ray-ban-display-glasses-and-neural-band/>.
 - [51] Microsoft. 2019. Hololens. <https://www.microsoft.com/en-us/hololens/>.
 - [52] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. 2025. Large Language Models: A Survey. *arXiv:2402.06196 [cs.CL]* <https://arxiv.org/abs/2402.06196>
 - [53] Yasir Abdelgadir Mohamed, Akbar Khanan, Mohamed Bashir, Abdul Hakim HM Mohamed, Mousab AE Adiel, and Muawia A Elsadig. 2024. The impact of artificial intelligence on language translation: a review. *Ieee Access* 12 (2024), 25553–25579.
 - [54] OpenAI. 2025. OpenAI API Documentation. <https://openai.com/api/>.
 - [55] Zhenhui Peng, Kaixiang Mo, Xiaogang Zhu, Junlin Chen, Zhijun Chen, Qian Xu, and Xiaojuan Ma. 2020. Understanding User Perceptions of Robot’s Delay, Voice Quality-Speed Trade-off and GUI during Conversation. In *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems*. 1–8.
 - [56] Jay Peters. 2023. Meta has reportedly sold 1 million Ray-Ban Stories glasses. <https://www.theverge.com/2023/12/5/23989359/meta-ray-ban-stories-sales-one-million>. <https://www.theverge.com/2023/12/5/23989359/meta-ray-ban-stories-sales-one-million>. Accessed: 2025-05-22.
 - [57] Proceedings on Privacy Enhancing Technologies. [n. d.]. Privacy Risks of Smart Glasses: A PETS 2023 Study. <https://petsymposium.org/popets/2023/popets-2023-0117.php>.
 - [58] Eman Ramadan, Arvind Narayanan, Udhaya Kumar Dayalan, Ros-tand AK Fezeu, Feng Qian, and Zhi-Li Zhang. 2021. Case for 5G-aware video streaming applications. In *Proceedings of the 1st workshop on 5G measurements, modeling, and use cases*. 27–34.
 - [59] Even Realities. 2025. Smart Glasses & Wearable Technology — Even Realities. <https://www.evenrealities.com/smart-glasses>.
 - [60] Nikolaos Schizas, Aristeidis Karras, Christos Karras, and Spyros Sioutas. 2022. TinyML for ultra-low power AI and large scale IoT deployments: A systematic review. *Future Internet* 14, 12 (2022), 363.
 - [61] Katie Seaborn, Norihisa P Miyake, Peter Pennefather, and Mihoko Otake-Matsuura. 2021. Voice in human-agent interaction: A survey. *ACM Computing Surveys (CSUR)* 54, 4 (2021), 1–43.
 - [62] Tuomo Sipola, Janne Alatalo, Tero Kokkonen, and Mika Rantonen. 2022. Artificial intelligence in the IoT era: A review of edge AI hardware and software. In *2022 31st Conference of Open Innovations Association (FRUCT)*. IEEE, 320–331.
 - [63] Silicon Republic Staff. 2025. Apple pivots from VR headsets to build AI smart glasses, says Bloomberg – Silicon Republic. <https://www.siliconrepublic.com/business/apple-vr-headsets-ai-smart-glasses-meta-ray-ban-vision-pro-bloomberg>. *Silicon Republic* (02 Oct 2025).
 - [64] Xiaoyu Sun, Xiaochen Peng, Sai Qian Zhang, Jorge Gomez, Win-San Khwa, Syed Shakib Sarwar, Ziyun Li, Weidong Cao, Zhao Wang, Chiao Liu, et al. 2024. Estimating power, performance, and area for on-sensor deployment of ar/vr workloads using an analytical framework. *ACM Transactions on Design Automation of Electronic Systems* 29, 6 (2024), 1–27.
 - [65] Temu. 2026. Smart Translation Glasses with Photochromic Lenses and AI Real-Time Translation. <https://www temu.com/smart-translation---photochromic-lenses--lenses-featuring-a-powerful-ai-translation-system-supporting-real-time-translation-and-multiple---with-an-8mp-hd-camera-eis-intelligent-image-g-601103532680933.html>.
 - [66] The Wireshark Team. [n. d.]. Wireshark: The world’s most popular network protocol analyzer. <https://www.wireshark.org/>. Accessed: 2025-09-12.
 - [67] Kurt Wagner and Tommaso Ebhardt. 2025. Meta Invests \$3.5 Billion in EssilorLuxottica in AI Glasses Push. <https://www.bloomberg.com/news/articles/2025-07-08/meta-invests-3-5-billion-in-essilorluxottica-in-ai-glasses-push>. *Bloomberg News* (2025).
 - [68] Xubin Wang, Zhiqing Tang, Jianxiong Guo, Tianhui Meng, Chenhao Wang, Tian Wang, and Weijia Jia. 2025. Empowering edge intelligence: A comprehensive survey on on-device ai models. *Comput. Surveys* 57, 9 (2025), 1–39.
 - [69] Yifan Wang, Minzhao Lyu, and Vijay Sivaraman. 2024. Characterizing User Platforms for Video Streaming in Broadband Networks. In *Proceedings of the 2024 ACM on Internet Measurement Conference*. 563–579.
 - [70] Shengqian Yang, Dacong Yan, and Atanas Rountev. 2013. Testing for poor responsiveness in Android applications. In *2013 1st international workshop on the engineering of mobile-enabled systems (MOBS)*. IEEE, 1–6.
 - [71] André Felipe Zanella, Antonio Bazco-Nogueras, Cezary Ziemlicki, and Marco Fiore. 2023. Characterizing and modeling session-level mobile traffic demands from large-scale measurements. In *Proceedings of the*

2023 ACM on Internet Measurement Conference. 696–709.

- [72] Zili Zhang, Yinmin Zhong, Yimin Jiang, Hanpeng Hu, Jianjian Sun, Zheng Ge, Yibo Zhu, Daxin Jiang, and Xin Jin. 2025. Disttrain: Addressing model and data heterogeneity with disaggregated training for multimodal large language models. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 24–38.

APPENDIX A: Voice AI Questions

1. What's the capital of France?
2. What is two plus two?
3. What color is the sky?
4. How many days are in a week?
5. What's the opposite of hot?
6. What is the current year?
7. Spell the word "glass."
8. What's the square root of sixteen?
9. Who wrote Romeo and Juliet?
10. What's the boiling point of water in Celsius?
11. What's your favorite color and why?
12. Describe a cat in one sentence.
13. What's something you can see outside?
14. Tell me a quick fun fact about space.
15. What's the best time to drink coffee?
16. Give me a synonym for "fast."
17. What's a common use for electricity?
18. Why do people sleep at night?
19. What's a good breakfast food?
20. What does AI stand for?
21. Explain how Wi-Fi works in simple terms.
22. How does a battery store energy?
23. Why do leaves change color in fall?
24. What happens when ice melts?
25. How does GPS know where you are?
26. Explain why airplanes can fly.
27. What causes an echo?
28. How does Bluetooth connect devices?
29. What's the difference between RAM and storage?
30. How do clouds form?
31. What's your name?
32. Can you remember what I just asked?
33. Repeat my last sentence.
34. What did I ask before that?
35. What did you just say?
36. If I say "banana," what do you say?
37. Say "ready" every time I stop talking.
38. I'll count to three — respond when I finish.
39. Tell me when you hear the word "test."
40. How quickly can you respond to this sentence?
41. What's the weather like today in New York?
42. How many minutes until the next hour?
43. Convert 10 meters to feet.
44. Set a timer for five seconds.
45. What's two plus two plus three?
46. How many letters are in the word "latency"?
47. What's faster: light or sound?
48. Tell me a random number between 1 and 100.
49. Finish this phrase: "Hello, my name is..."
50. Say "response complete" when you're done talking.

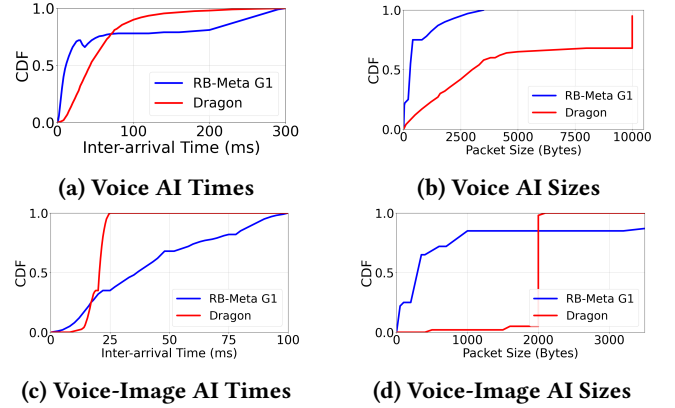


Figure 21: Inter-packet arrival times and packet sizes for P-DL/G-DL audio responses during Voice AI and Voice-Image AI on RB-Meta G1 and Dragon. RB-Meta G1 favors smaller, more frequent packets, while Dragon returns larger, burstier chunks.

APPENDIX B: Further Insights on AI interactions

Packet-level response behavior. We provide further insight into the voice response by obtaining inter-packet arrival times and packet sizes of the responses. For voice AI, Figure 21a shows that **RB-Meta G1** has a median inter-packet arrival time of 42 ms and a max of 315 ms, while **Dragon** has a median inter-packet arrival time of 21 ms and a max of 277 ms. This is most likely because the actual voice data being exchanged by Meta is quite large, explaining the larger inter-packet arrival times at the higher end. Further supporting this, we see that in figure 21b that **RB-Meta G1** has a median packet size of 315 bytes and a max of 3337 bytes, while **Dragon** has a median packet size of 3355 bytes and a max of 14308 bytes. This supports our finding that **RB-Meta G1** transmits fewer bytes per packet in a shorter amount of time, while **Dragon** has lower packet arrival times and transmitting more bytes per packet.

For Voice-Image AI responses, Figures 21c and 21d show that the packet-level behavior mirrors the voice-only pattern but with a narrower spread. **RB-Meta G1** continues to favor frequent, small packets while **Dragon** delivers larger, more bursty transfers at shorter intervals. This reflects their fundamentally different audio delivery strategies, **RB-Meta G1** streams responses in a steady, fine-grained flow while **Dragon** batches data into larger chunks, consistent with its use of separate cloud APIs for each pipeline stage.

APPENDIX C: Live-Streaming Resolution Levels

This appendix holds the resolution evidence for the phone-side decode-upscale-re-encode loop reported in §4.1: the per-configuration G-UL and P-UL resolution distributions (Figure 22) and the mapping from resolution IDs to pixel dimensions (Table 3).

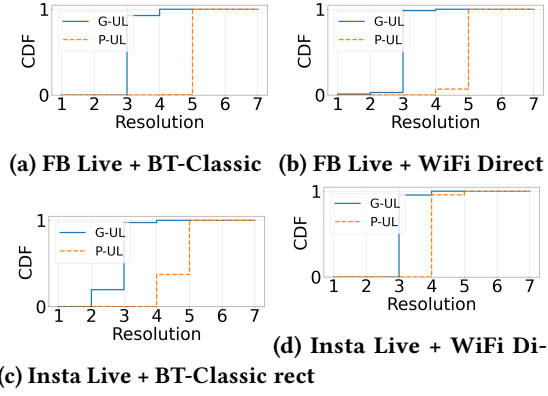


Figure 22: Video resolution levels over G-UL vs. P-UL during live streaming from RB-Meta G1 (resolution IDs mapped in Table 3). The glasses send low-resolution video (IDs 3–4) while the phone uploads at higher resolution (IDs 5–6) across all four configurations.

Table 3: Mapping of resolution IDs used in Figure 22.

Mapping ID	Resolution Group
1	288x512
2	360x640
3	432x768
4	504x896
5	720x1280
6	1080x1920
7	1836x3264
8	2268x4032

Resolution levels over G-UL vs. P-UL. Figure 22 compares the resolution levels of the video transmitted from G-UL versus the resolution ultimately uploaded from P-UL. The numeric "resolution identifiers" on the X-axis correspond to the concrete pixel resolutions listed in Table 3. Across all scenarios, over both Bluetooth Classic and Wi-Fi Direct, the glasses consistently send a lower, tightly clustered resolution to the phone, while the phone uploads a much higher-resolution stream to the platform.